

PortSentinel Nexus System Architecture and Data Flow

Technical briefing for prototype understanding and showcase preparation

SYSTEM	Port-resilience decision-support prototype for Deendayal Kandla Port
PROBLEM SCOPE	L1 Container Congestion Predictor and Port Operations Optimiser plus U1 Power Outage Prediction and Grid Equipment Failure Advisor
IMPLEMENTATION STATUS	Working local prototype with simulated port operations, live external weather, deterministic scenario evaluation, supplied model inference and human approvals
REPORT BASIS	Repository code and configuration reviewed on 15 September 2026; backend 17 tests passed and frontend TypeScript validation passed

Main conclusion

The system connects infrastructure condition and weather to handling capacity, vessel queues, ranked response scenarios and an auditable human decision. The calculation path is owned by the Python backend; AI is restricted to arranging already-authoritative facts for explanation and cannot change scores, recommendations or approvals.

1 System Purpose and Problem Statement Coverage

PortSentinel Nexus addresses the operational question behind both problem statements: when equipment or power health deteriorates under changing weather, what downstream congestion is likely, which response performs best under the same assumptions, and what should a human operator approve? Deendayal Kandla Port is the reference setting. Port records are simulated because no private telemetry, AIS feed or terminal operating system is connected.

Feature traceability

PS requirement	Implemented capability	Technology	Current evidence and boundary
L1 Understand port congestion	0 to 48 hour berth queue forecast, demand capacity index, utilization, waiting count and vessel delay	Python deterministic queue engine plus React SVG chart	Implemented. It is an engineering simulation, not a trained congestion model or calibrated port forecast.
L1 Optimise port operations	Five comparable responses: do nothing, repair, crane reassignment, vessel shift, combined repair and rebalance	ScenarioEngine runs every option through the same CongestionPredictor	Implemented with feasibility gates and a transparent score. Repair time, cargo compatibility, pilots and tugs require operator confirmation.
L1 Operational visibility	Home, interactive Live Port View, Alerts, Approvals, Congestion, Weather, Assets, Vessels, Berths and Cranes	React 19, TypeScript, Vite and Three.js	Implemented as a local web prototype. The 3D quay is illustrative and the home photograph is archival, not a live camera.
U1 Equipment failure or outage risk	Supplied calibrated classifier returns numeric class 1 probability and threshold band for assets and cranes	scikit-learn CalibratedClassifierCV over XGBoost	Implemented from the supplied artifact without retraining. Class 1 meaning and original feature formulas are not confirmed, so the UI avoids claiming a verified failure probability.
U1 Weather aware infrastructure risk	Current Open-Meteo conditions are combined with an asset health proxy to form the complete 16 feature vector	HTTPX weather integration and Python feature bridge	Implemented for simulated assets. Weather values are direct; the weather stress formula and machine-sensor interpolation are derived demo assumptions.
U1 Failure advisor	Asset to crane to berth to vessel dependency chain, criticality index, alert and repair response	DependencyEngine, AlertEngine and ScenarioEngine	Implemented as deterministic decision support. Criticality is an index, not a probability.
Human authority	Approve, choose an evaluated alternative and modify, or reject; persist plan and audit	FastAPI, Pydantic validation and SQLite transactions	Implemented. No equipment command, dispatch or autonomous action is issued.

What the prototype proves

- A single operational chain can connect asset condition, dependencies, berth capacity, vessel delay, response comparison and approval.
- The same reproducible input snapshot can drive the forecast, recommendation, explanation and audit record.
- External weather and AI-provider outages degrade explicitly instead of being disguised with fake live data or invented model output.

2 Technology Stack and Why It Is Used

Layer	Technology	Role in this system	Why this choice fits the prototype
User interface	React 19 and TypeScript 5.8	Navigation, operational dashboards, registers, scenario comparison and decision forms	Typed reusable components support a multi-module interface and reduce contract mistakes between UI and API.
Build and local routing	Vite 6	Development server, production bundle and /api proxy to port 8000	Fast startup and a simple same-origin frontend development path.
3D visualization	Three.js 0.180	Illustrative quay, vessel and crane scene in Live Port View	Makes dependencies and operational context easy to demonstrate without claiming a surveyed digital twin.
UI icons and charts	Lucide React and native SVG	Navigation icons, status cues and congestion lines	Lightweight presentation without a separate charting framework.
API and orchestration	Python 3.12, FastAPI 0.115 and Uvicorn	Typed HTTP endpoints, pipeline orchestration and OpenAPI documentation	Keeps validation, model inference and operational calculations in the same process for a local prototype.
Validation	Pydantic 2.11	Rejects malformed records, invalid ranges, stale decisions and incomplete model inputs	Makes API contracts explicit and testable.
Persistence	SQLite with WAL mode	Registers, revision, reports, decisions, audit events and weather snapshots	Zero-service local database with transactions and durable demo state.
Machine learning	scikit-learn 1.6, XGBoost 3.4, NumPy and pandas	Loads the supplied calibrated model and performs ordered 16-feature inference	Preserves the provided model artifact and its calibration rather than substituting a new model.
External HTTP	HTTPX 0.28	Open-Meteo and optional LLM-provider requests	Supports timeouts, test transports and explicit HTTP failure handling.
Quality checks	pytest, TypeScript compiler and Vite build	Backend integration coverage and frontend contract checks	Makes the showcase flow reproducible before presentation.

Runtime topology

Browser React UI on 127.0.0.1:5173	Vite proxy Forwards /api	FastAPI Python on 127.0.0.1:8000	Engines Model, weather, dependencies, queues, scenarios	SQLite Local durable state and audit
---------------------------------------	-----------------------------	-------------------------------------	--	---

Deployment note. This is a two-process, loopback-only demonstration. It has no production deployment, multi-user identity system, role-based access control or external database service.

3 End to End Data Flow

1 Input Registers, simulation clock and weather	2 Risk 16 feature model inference	3 Impact Dependencies and berth queues	4 Options Scenario simulation and ranking	5 Decision Human review, plan and audit
---	---	--	---	---

Detailed request and analysis sequence

1. The browser requests GET /api/snapshot through the Vite /api proxy. The frontend sends an explicit X-PortSentinel-Client header; an optional operator token is added only for protected mutations.
2. FastAPI asks Store for a consistent copy of Assets, Cranes, Berths, Vessels, the simulation clock, state revision and prior decisions.
3. WeatherService requests current and hourly Open-Meteo data for 23.03 N, 70.22 E. It validates ranges, converts wind to knots and visibility to kilometres, labels timestamps and caches a successful snapshot for ten minutes.
4. For each simulated asset or crane, the pipeline forms a complete 16-feature input. Weather values come from the provider; machine features are interpolated from the asset health score. Operator-entered records must supply their own already-engineered feature object or inference is marked unavailable.
5. FailureRiskModel verifies the artifact checksum, loads its original embedded XGBoost model through a compatibility recovery path, preserves sklearn calibration and returns numeric class probabilities plus the saved threshold band.
6. DependencyEngine traverses asset supply relationships, crane assignments, berth demand and vessels. It computes workload exposure, six-hour arrival pressure, substitutes and a heuristic criticality index.
7. CongestionPredictor calculates effective crane capacity using equipment health, supply health, restrictions and current wind derating. A deterministic per-berth queue produces vessel start, finish and delay values plus a six-hourly 0 to 48 hour port series.
8. AlertEngine raises infrastructure-congestion or weather-exposure alerts when configured thresholds are crossed.
9. ScenarioEngine modifies a copy of the same snapshot for each intervention, reruns the same queue predictor, checks feasibility and ranks feasible options by total delay plus congestion plus effort.
10. The pipeline saves an immutable report whose ID is derived from state revision, weather status and time, engine version and model metadata. The frontend renders the same report in Live Port View, Alerts, Congestion and Approvals.
11. On request, the configured LLM receives only authoritative fact IDs and text. It returns an order and selection of IDs. The backend validates those IDs and assembles exact stored text; failure produces a labeled deterministic explanation.
12. An operator approves, modifies by selecting another evaluated scenario, or rejects. SQLite records the decision transactionally with actor, timestamp, scenario snapshot, model outputs, accepted demo plan and audit event. No equipment command is produced.

Consistency controls

- If-Match revision is required for register writes, preventing a stale browser from overwriting a newer port state.
- Approval checks the current report ID and current revision; a changed register or weather snapshot makes the old recommendation stale.
- A unique report decision and request ID provide single-decision and idempotency protection.

4 Data Inputs Transformations and Outputs

Data group	Source	Key fields	Transformation	Consumed by
Port registers	Seeded JSON or operator edits	Assets, cranes, berths, vessels, health, status, rates, cargo, ETA, dimensions, supply asset	Pydantic validation, SQLite persistence and revision increment	Dependencies, queues, UI and audit
Weather	Open-Meteo	Temperature, humidity, precipitation, weather code, wind, visibility and hourly forecast	Range validation, units, direction label, severity thresholds, cache and stale status	Model bridge, queue derating, alerts and Weather UI
Model features	Operator supplied, or derived for simulated records	Eight machine features, three type flags and five weather features	Exact feature-set and finite-value checks; ordered pandas frame	Calibrated classifier
Model output	Supplied artifact	Class 0 and class 1 probability, predicted class and saved band	Positive-class semantic guard	Dependencies, queue capacity, facts and audit
Dependency result	Deterministic traversal	Affected cranes, berths, vessels, cargo exposure, substitutes and criticality	Weighted index from exposure, arrival pressure and substitute availability	Alerts and scenario target selection
Congestion result	Deterministic queue	Berth capacity, vessel schedule, delay, utilization, queue and 48 hour series	Same simulation for baseline and all scenarios	Alerts, ranking, charts and approvals
Decision	Human operator	Report, scenario, status, comment and request ID	Transactional validation and immutable scenario snapshot	Accepted demo plan, history and audit

The 16 model features

Machine and engineered values: Air_temperature, Process_temperature, Rotational_speed, Torque, Tool_wear, Temp_diff, Torque_x_Speed and Power_est.

Equipment type flags: Type_H, Type_L and Type_M, with exactly one set to 1.

Weather values: Ambient_temp_C, Wind_speed_kmh, Rainfall_mm, Storm_flag and Weather_stress_index.

Important semantic boundary

The supplied artifact proves numeric class prediction, not yet a verified equipment-failure label. Until the training owner confirms that class 1 means failure and provides the original preprocessing formulas and units, the operational simulator primarily relies on health exposure where confirmed failure risk is unavailable. The report and UI preserve this distinction.

5 Core Calculation Logic

Equipment and weather inference

- The model file is checksum-pinned to 8926d985a056f01f57d3f981e3b67bfed7d16d172c0859f43d6d123b90acb28c.
- The loader accepts only expected serialized types and recovers the original embedded XGBoost model bytes from an incompatible saved memory snapshot.
- Simulated machine values are linearly interpolated between healthy and stressed demo anchors using health 0 to 100.
- Weather stress index is clipped between 0 and 1 using a derived formula. It matches the two supplied demo anchors but is not confirmed training preprocessing.

Handling capacity and queue

Effective crane capacity starts from handlingRate, reduces it by the minimum of crane and supplying-asset health, optionally reduces it for confirmed model risk, applies a restricted-status factor and applies a current-wind factor. A supplying asset in Maintenance removes dependent crane capacity.

Queue logic orders vessels by arrival, priority and ID. A vessel starts when it has arrived and its berth becomes free. Cargo units divided by available cargo-units-per-hour gives handling duration. Incompatible dimensions or zero capacity create a lower-bound delay at the 48-hour window.

Dependency criticality

Criticality = weighted exposure from risk or health, cargo workload, arrivals within six hours and substitute availability, capped at 100. This produces a comparison index for attention routing; it is not a probability or certified safety score.

Scenario ranking

Score = total vessel delay hours + 0.1 x peak congestion index + 2 x resource effort units. Lower is better among feasible scenarios.

Scenario	What changes in the simulation	Feasibility or assumption
S0 Do nothing	Keeps current records and monitoring	Baseline exposure remains.
S1 Repair	Restores target asset and dependent crane health to 96 and Available	Repair is optimistically complete before the window.
S2 Reassign crane	Moves an available substitute crane to the target berth	Donor berth, travel and lifting compatibility require checks.
S3 Shift vessel	Moves the next waiting or inbound vessel to another dimension-compatible berth	Cargo compatibility, pilot and tug availability require checks.
S4 Repair and rebalance	Combines repair and eligible crane reassignment	Uses both sets of assumptions and resources.

6 External Services Assets and Why They Exist

External item	Purpose	Data sent or received	Failure behavior	Current status
Open-Meteo	Provide current and hourly weather for the Kandla reference coordinates without a prototype API key	Receives latitude, longitude and requested fields; returns modeled weather	Ten-minute cache, one-minute retry backoff, stale snapshot or explicit unavailable state	Integrated and tested. It is an external weather model, not an on-site sensor feed.
SambaNova chat-completions	Optional AI ordering of backend facts into an operator briefing	Receives report ID plus authoritative fact IDs and text; returns fact_ids JSON	Grounding validation; deterministic backend explanation on any error	Default configured provider and model Meta-Llama-3.3-70B-Instruct. A real completion is not verified because the provider returned payment-method-required.
IBM Bob API adapter	Alternative explanation provider when an actual inference contract is available	Same constrained fact-selection contract	HTTPS and protocol validation plus deterministic fallback	Implemented but endpoint, model and credentials are not verified.
Groq adapter	Additional selectable explanation provider	Same constrained fact-selection contract	Dedicated credential isolation and deterministic fallback	Implemented in code and covered by mocked tests; not listed in the current environment template.
Archival port photograph	Ground the Home page visually in Deendayal Port	Static local image with an attribution link	UI remains functional without live visual data	Not a camera feed. Redistribution rights should be confirmed before public distribution.
No connected port systems	Explicit absence rather than a hidden dependency	No AIS, SCADA, IoT telemetry, terminal operating system, grid control or dispatch interface	Simulation continues with labeled seeded records	Deliberate prototype boundary.

Why AI is constrained

The LLM is not the optimiser. All risk values, dependencies, queues, scenario metrics and ranking are created before the LLM call. The LLM can select and order known fact IDs only. The backend then renders exact fact text and forcibly includes recommendation, authority and limitations. This preserves reproducibility and keeps approval with the operator.

Credential and network boundary

- Provider keys remain in backend environment variables and are never placed in VITE-prefixed frontend variables.
- SambaNova, Groq and Bob each use separate credentials; the code does not forward one provider's key to another.
- LLM requests require HTTPS. HTTP redirects are disabled for weather and provider requests.

7 Application Modules and API Surface

UI module	What the operator sees	Primary backend data
Home	Archival port image, system introduction and entry to operations	Snapshot status and basic context
Live Port View	Interactive illustrative 3D quay with selectable vessels, cranes and berths	Registers and congestion forecast
Alerts	Cause, dependency chain, impact and ranked response cards	Alerts, risks, dependencies and scenarios
Approvals	Scenario comparison, optional AI explanation, approve modify reject workflow, accepted plan and history	Immutable report, decisions and audit
Congestion	Current and peak index, 48-hour chart and berth-level outlook	Congestion series and scenario series
Weather	Current conditions, 24-hour outlook, severity and affected assets	Normalized Open-Meteo snapshot
Assets Vessels Berths Cranes	Searchable editable operational registers	SQLite records and revision
Settings	Service state, provider status, simulation controls and optional operator token	Health metadata, revision and simulation endpoints

API groups

Group	Endpoints	Purpose
Read model and state	GET /api/health, /snapshot, /port/live, /model/metadata	Service status, consistent workspace and model interface
Operational analytics	GET /api/congestion, /dependencies, /alerts, /scenarios	Calculated outputs for dashboards and review
Registers	GET POST /api/assets vessels berths cranes; PUT /api/{domain}/{id}	Read and revision-protected record changes
Weather and inference	GET /api/weather; POST /api/model/predict	Refresh normalized weather and explicitly run complete feature inference
Decision flow	GET POST /api/approvals; GET /operations, /audit, /recommendations/{id}	Review, persist and retrieve plans and evidence
AI explanation	POST /api/explanations	Request provider-backed fact ordering or deterministic fallback
Simulation	POST /api/simulation	Advance, degrade, repair or reset the seeded state

8 Persistence Security and Human Control

SQLite records

Table	Stored information	Reason
records	Current Assets, Cranes, Berths and Vessels as JSON	Durable editable demo registers
meta	Simulation clock and revision	Consistent recalculation and stale-write protection
reports	Immutable analysis snapshot by report ID	Reproducible recommendation and explanation context
decisions	One human decision per report plus unique request ID	Atomic approval, modification or rejection
audit	Timestamped workspace, record, simulation, explanation and decision events	Traceability for the showcase
weather	Successful external snapshots	Provider provenance and later review

Controls that are implemented

- Input schemas forbid unexpected fields and constrain IDs, ranges, timestamps, vessel identifiers and decision status.
- Mutations require the UI client header; an optional shared operator token is compared securely on the server.
- SQLite transactions roll back failed writes and use WAL mode for the local demo.
- Reports are immutable snapshots and decisions store the selected scenario plus the model results used at approval time.
- Reset replaces simulated registers but deliberately retains audit and decision history.

Controls that are not implemented

- No real authentication provider, per-user identity, role-based authorization, encryption-at-rest design or secrets manager.
- No production network boundary, reverse proxy, TLS termination, rate limiting or centralized monitoring.
- No operational actuation; accepted plans are stored as `not_executed` and require human follow-through.

9 Current Build Status Evidence and Limitations

Area	Status on 15 September 2026	Evidence
Backend	Pass	17 focused pytest tests passed in 3.80 seconds; one Starlette AnyIO deprecation warning.
Frontend contracts	Pass	npm run typecheck completed successfully.
Model loading	Pass with compatibility recovery	Checksum-pinned supplied artifact loads and produces tested healthy and stressed class-1 probabilities.
Weather	Implemented	Normalization, stale fallback, range checks and live-weather-to-feature integration are covered by tests.
LLM adapters	Mock validated	Bob, SambaNova and Groq credential isolation and grounded fact-ID behavior are tested with mock transports.
Real SambaNova inference	Blocked externally	Provider response indicated payment method required; deterministic explanation remains available.
Submission package	Incomplete	Team metadata, final recording, presentation and live demo link remain to be supplied.

Known limitations that should be stated during the demo

- Operational records, positions, arrivals and equipment condition are simulated.
- The 3D port is illustrative; it is not a surveyed digital twin.
- No AIS, SCADA, IoT, terminal operating system or real grid data is connected.
- The supplied model's class-1 meaning, units and original preprocessing are not confirmed.
- Weather stress and machine-sensor features for simulated assets use transparent derived assumptions.
- Queue and scenario scores are engineering heuristics, not certified forecasts, costs or safety decisions.
- AI explanation availability does not affect calculations or operator authority.
- The local shared-token gate is not production authentication.

Readiness for the next showcase step

The best demonstration path will follow the system's real causal chain: Home to Live Port View to a controlled degradation event, then Alerts, Congestion, scenario comparison, optional AI explanation, Approvals, and finally the persisted plan and audit evidence. That flow can now be scripted precisely because every screen maps to the same immutable backend report.

Appendix Repository Evidence Map

Concern	Primary implementation files
Problem and intended product	docs/problem-statement.md; docs/solution-overview.md
API and route protection	src/backend/app/main.py; src/backend/app/schemas.py
Pipeline and facts	src/backend/app/pipeline.py
Model loading and weather feature bridge	src/backend/app/failure_model.py; src/backend/models/model1_failure_risk_weather.pkl
Weather provider	src/backend/app/weather.py; src/backend/app/config.py
Dependencies, congestion, alerts and scenarios	src/backend/app/dependencies.py; congestion.py; alerts.py; scenarios.py
Persistence and human decisions	src/backend/app/store.py
Seeded demonstration state	src/backend/app/simulation.py; src/backend/app/fixtures/port.json
AI providers and grounding	src/backend/app/llm.py
Frontend API contract	src/frontend/src/services/portService.ts
Operational screens	src/frontend/src/App.tsx; src/frontend/src/components
Executable validation	src/backend/tests/test_pipeline.py; src/frontend/package.json

Interpretation guide

Implemented means executable code exists in the repository and is represented in the current frontend or API. **Mock validated** means behavior is covered with a controlled transport but a successful real provider call is not claimed. **Boundary** identifies a capability deliberately excluded from this prototype or awaiting source confirmation.